

# An Intelligent Predictive Resource Management Framework for Cloud-Native Systems Based on Large Language Models

1<sup>st</sup> Hao Yang\*

*Information and Communication Company  
State Grid Gansu Electric Power  
Lanzhou, China  
yangh4o509@gmail.com*

2<sup>nd</sup> Jianyong Gao

*Information and Communication Company  
State Grid Gansu Electric Power  
Lanzhou, China  
gaojianyong0806@gmail.com*

3<sup>rd</sup> Shaobo Liu

*Information and Communication Company  
State Grid Gansu Electric Power  
Lanzhou, China  
liushaobo0806@gmail.com*

4<sup>th</sup> Yipeng Xiong

*Information and Communication Company  
State Grid Gansu Electric Power  
Lanzhou, China  
xiongyip0806@gmail.com*

5<sup>th</sup> Rui Dong

*Information and Communication Company  
State Grid Gansu Electric Power  
Lanzhou, China  
dongrui0806@gmail.com*

**Abstract**—As cloud-native architectures gained ubiquity, microservices enhance system flexibility. But because of their distributed nature, they are also confronted with intricate inter-service dependencies. The result is that fault identification is challenging and conventional reactive resource capacity expansion strategies have response lags, compromising operations and maintenance. As a response to such challenges, this paper puts forward an intelligent predictive resource management framework for cloud-native systems using a large oracle model. This architecture amalgamates open source libraries such as OpenTelemetry, Prometheus, and Loki to capture and process three types of observability data consistently: logs, metrics, and tracebacks. A visualization dashboard is also built to offer improved operational visibility. The novelty in this paper relies on adding a predictive autoscaling framework with the use of the LLM. Six scenarios for loading are proposed, merging the LLM with mathematical classification definitions. Based on the integration of text and image information, a bimodal analysis method is employed to intelligently identify load patterns and select the optimal prediction model. Experiments indicate that compared with previous practices, this approach significantly improves prediction accuracy and efficiency in resource utilization, and it provides a new technical means for cloud-native system operation and maintenance.

**Index Terms**—*Large Language Model, Predictive Autoscaling, Cloud-Native, Multimodal Learning, Intelligent Resource*

This research was supported by State Grid Gansu Electric Power Company 2025 Scientific Research Cost Project (522723250008) and Research and Application of Information Security Key Technologies for Industrial Control Systems(23YFGA0010)

## Management

### I. INTRODUCTION

With the advancement of digital transformation, cloud-native architecture has become one of the top choices for modern information systems. By breaking down large applications into multiple independent microservice units, microservice architecture has significantly improved the flexibility and scalability of the system [1,2]. However, the complexity of distributed systems also brings many challenges to the system, such as complex dependencies between microservices and diverse fault propagation paths, which traditional operation and maintenance methods are no longer able to cope with [3].

Currently, the operation and maintenance of cloud-native systems faces two core issues: observability challenges and the need for intelligent resource management. Tracing, metrics, and logs are the three core dimensions of observability [4]. Information is usually collected and stored independently, resulting in data silos and making it difficult to obtain a global overview of the system [5,6]. At the same time, traditional responsive scaling strategies, due to their responsive nature, have fixed delays and are unable to respond to dynamic load changes in a timely manner, resulting in resource waste or reduced service quality [7].

In recent years, large language models (LLMs) have demonstrated strong capabilities in identifying patterns and making

judgments in many fields, which has brought new breakthroughs to traditional operations and maintenance management. This paper focuses on how to use the understanding and perception capabilities of large language models in the resource scheduling of cloud-native systems, so that the system can take action in advance instead of just waiting for problems to occur. This paper designs a cloud-native intelligent predictive resource management framework based on large language models.

The main contributions include: (1) building a structure for uniformly collecting and processing observable data, integrating scattered information and solving the problem of data being independent of each other (2) sorting out six common load operating states and describes their characteristics mathematically to help the system more accurately select the right prediction method (3) attempting to introduce large language models into the cloud-native resource management process for the first time, realizing dual-channel recognition of load types by simultaneously analyzing text descriptions and chart images.

## II. KUBERNETES INTELLIGENCE OVERVIEW

As companies continue to undergo digital transformation, cloud-native architecture is one of the most common technology directions in today's information systems. Container technology plays a crucial role in enterprise business systems due to its rapid startup, scalability, portability, and efficiency in resource utilization [2,8-11]. By migrating production workloads to container platforms, companies can significantly enhance business delivery speed while maximizing infrastructure utilization [11]. Kubernetes, the open-source system for container orchestration developed by Google, has become the de facto standard for cloud-native infrastructure [12-14] due to its cross-host scheduling, lightweight design, and extensibility.

In a Kubernetes cluster, elastic scaling is enabled via the dynamic scaling of node capacity, Pod replicas, or resource quotas [15]. The structure of the system workload is implemented in two layers: the infrastructure layer (Node nodes) and the application layer (Pods managed by Deployment, StatefulSet, and DaemonSet) [16]. Horizontal scaling, which scales Pod replicas from monitored metrics, is the most widely used auto-scaling technique. However, such reactive legacy behavior is marred with serious pitfalls: inherent latency in scaling operations can lead to traffic congestions during traffic spikes, which contribute to cascading failures and reduced service .

Unlike reactive scaling, our research is necessity-driven to evolve from event-based operations to proactive resource management fueled by AI. The recent advancements in large language models for natural language understanding, time-series modeling, and multi-modal reasoning have shown strong ability in capturing complex workload patterns and proposing best predictive strategies. Accordingly, this paper explores the integration of LLM cognition into Kubernetes resource scheduling for developing intelligent predictive resource management framework based on large language models that

transforms the traditional "detect-then-react" pattern into a "predict-then-prepare" paradigm. This transition not only improves system responsiveness in dynamic loads but also offers a channel for intelligent cloud-native operations with AI-aided decision-making and adaptive execution.

## III. INTELLIGENT FRAMEWORK DESIGN AND IMPLEMENTATION

The cloud platform itself has the characteristics of elastic scalability and distribution, which significantly improves the flexibility and stability of cloud-native architecture in applications. However, due to the complex system structure and the many call relationships between services, traditional monitoring methods are increasingly struggling in data collection and problem analysis, making it difficult to see the true status of the system in a timely manner.

To address these problems, this paper refers to the closed-loop control idea of MAPE (Monitor-Analyze-Plan-Execute) [17,18] and builds a cloud-native intelligent predictive resource management solution combined with a large language model, allowing the entire operation and maintenance process to perceive, analyze, make decisions and execute by itself, forming a complete automatic cycle. The focus of this system is to add a pattern recognition and predictive scaling mechanism driven by a large language model. It can learn common load change patterns from historical data and predict the next resource demand in advance, instead of waiting until the indicators exceed the standard before taking action. This method overcomes the problem that traditional scaling methods are always slow, and transforms the old model of only dealing with problems when they occur into a new method that can predict and adjust in advance.

### A. Intelligent Monitoring Stage

The system's monitoring phase is tasked with collecting various observable information within the cloud-native environment. Standard scaling methodologies typically employ manually set thresholds to recognize anomalies and respond only after problems have arisen. Standard scaling is different in that it repeatedly collects system running information and analyzes the information to find both visible and invisible fault patterns. Instead of having to respond after receiving notification, potential problems are found ahead of time, shifting from reactive response to proactive forecasting. This quantifiable data has mostly three origins, which include logs, metrics, and tracing.

Logs capture the details of every system activity, what was run, whether it succeeded, and when it was executed. Metrics are ordered-in-time sequences of values that report status such as CPU and memory use, which provide visibility into patterns of resource utilization. Tracing captures the entire end-to-end path of a request making its way through multiple services, using spans to represent the time spent in each step and exceptions. It is needed to discover performance bottlenecks.

The system makes use of Prometheus as the underlying metrics collection component, which is utilized for collecting

and storing time-series data and querying via the PromQL language. OpenTelemetry Collector is deployed as a Daemon-Set; it gathers telemetry data and pushes trace data to Jaeger. Log collection is deployed based on the Loki architecture. Here container logs are gathered by the Promtail component, log integrity is verified by the Distributor component, chunks of data are constructed and compressed by the Ingester component, and both real-time and historical data are queryable through the Querier component.

### B. Intelligent Analysis Stage

The system processes and reconciles the multi-dimensional observability information collected during the analysis phase and inputs to derive operationally relevant details from raw data. The system improves its functionality related to data analysis with the addition of machine learning algorithms and multi-layered architectural analysis.

With PromQL, the system ingests metrics scraped by Prometheus in real-time, multi-dimensionally aggregates data, and captures performance variations accurately through a dynamic threshold-based anomaly algorithm. The system employs LogQL to enable efficient log retrieval as well as pattern discovery. For distributed tracing, the system employs a service dependency graph model built atop Jaeger. Through calculating causality relationships and temporal properties among spans, it provides exact localization.

### C. Intelligent Planning Stage

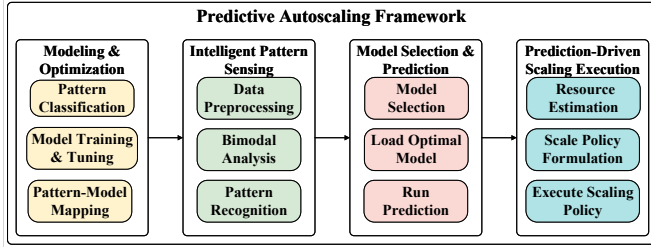


Fig. 1. Predictive Autoscaling Framework

The planning stage offers a predictive auto-scaling mechanism based on large language models, which combines LLMs and mathematical pattern classification methods to achieve intelligent identification of six workload patterns and optimal prediction model selection.

The framework defines six typical workload patterns, expressed as follows:

$$P_t = \sum_{k=1}^K A_k \cdot \sin\left(\frac{2\pi t}{T_k} + \phi_k\right) + B + N_t \quad (1)$$

where  $P_t$  is the number of Pods at time  $t$ ;  $A_k$ ,  $T_k$ , and  $\phi_k$  are the amplitude, period, and phase shift of the  $k$ -th harmonic;  $B$  is the baseline Pod count;  $N_t$  denotes the noise component.

$$P_t = S \cdot \sin(2\pi h_t/24) + B + G \cdot f(t) + N_t \quad (2)$$

where  $S$  is the amplitude of periodic components;  $h_t$  is the time variable in hours;  $G$  represents the growth rate;  $f(t)$  is the growth function; other symbols are defined as in (1).

$$P_t = B + \sum_i B_i \cdot g(t - t_i, d_i) \cdot 1 + S \cdot \sin(2\pi h_t/24) + N_t \quad (3)$$

where  $B_i$  is the magnitude of the  $i$ -th burst event;  $t_i$  and  $d_i$  are its start time and duration, respectively;  $g(t, d)$  is a decay function; 1 is the indicator function.

$$P_t = \begin{cases} P_{\text{high}} + N_t^{\text{high}}, & S_t = 1 \\ P_{\text{low}} + N_t^{\text{low}}, & S_t = 0 \end{cases} \quad (4)$$

where  $S_t$  is the binary state at time  $t$ ;  $P_{\text{high}}$  and  $P_{\text{low}}$  are the Pod counts during active and inactive states, respectively;  $N_t^{\text{high}}$  and  $N_t^{\text{low}}$  denote the noise terms under corresponding states.

$$P_t = S \cdot \sin\left(\frac{2\pi h_t}{24}\right) + \sum S_j \cdot e^{-(t-t_j)^2/2\sigma_j^2} + B + N_t + \sum T_i \cdot 1_{t \geq c_i} \cdot (t - c_i) \quad (5)$$

where  $S_j$ ,  $t_j$ , and  $\sigma_j$  denote the magnitude, occurrence time, and width of the  $j$ -th spike;  $T_i$  and  $c_i$  represent the slope and change point of the  $i$ -th trend segment; other symbols are defined as above.

$$P_t = B_{\text{base}} + L_t \cdot S_{\text{step}} + S \cdot \sin(2\pi h_t/24) + N_t \quad (6)$$

where  $B_{\text{base}}$  is the baseline workload level;  $L_t$  is the step-level indicator at time  $t$ ;  $S_{\text{step}}$  is the magnitude of each step change; other symbols are defined as in (1) and (2).

As illustrated in Figure 1, there are four steps in the prediction process: Pattern modeling optimization generates datasets and trains optimal prediction models; Intelligent pattern sensing employs dual-modal analysis to convert time-series data into images and text as inputs to LLMs for enhancing recognition accuracy; Pattern selection and prediction stages where LLMs identify workload patterns and provide best prediction models; Prediction-driven execution stages generate scaling strategies based on prediction outputs.

### D. Intelligent Execution Stage

The execution stage implements predictive resource scheduling based on the supervision of prediction framework outputs. As shown in Figure 1, the system integrates forecasted future resource requirements with node real-time measurements in the current time, calculates optimal resource sizes, and constructs and imposes corresponding horizontal scaling strategies for predictive provision of resources. Such prediction-supported execution process masterfully removes the latency shortcomings of traditional reactive scaling and significantly improves system responsiveness and resource utilization effectiveness.

## IV. EXPERIMENTAL DESIGN AND ANALYSIS

This chapter strictly verifies the effectiveness of the large language model-based predictive auto-scaling framework through extensive experiments.

In order to verify the validity of large language models in recognizing workload patterns, comparative experiments are performed with mainstream models such as Gemini-2.5-Pro, Qwen3, and DeepSeek Reasoner in this study, and the effect of text and image dual-modal analysis on recognition accuracy is explored.

As shown in Figure 2, experiment results confirm that dual-modal analysis-based large language models possess significantly higher average accuracy in workload pattern recognition tasks compared with single text-modal methods. The dual-modal method incorporates visual characteristics of time-series data visualization and text description features and can better capture more comprehensive workload pattern attributes, thus improving recognition accuracy. Such innovation corroborates technical excellence in multi-modal learning for time-series pattern recognition.

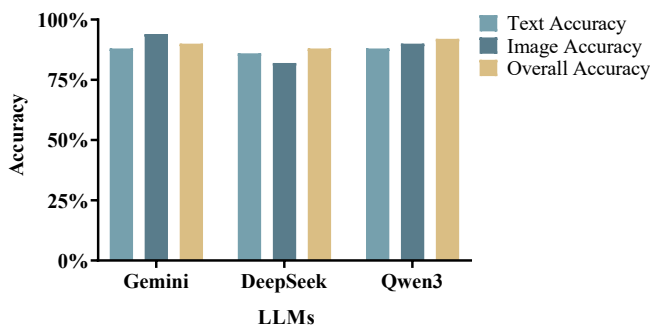


Fig. 2. Evaluation of Different Analysis Methods

## V. CONCLUSION

This paper addresses the key challenges of cloud-native system management by presenting and implementing a smart system management with large language models. Our approach surpasses traditional monitoring software that can only actively detect known bugs through pre-defined thresholds by creating an active fault localization and discovery system through the integration of three key observability axes: log analysis, metric monitoring, and distributed tracing, which significantly enhances operators' situational awareness of the system as well as diagnostic efficiency.

The primary technical contributions of this study are embodied in three aspects:

(1) We designed and deployed a unified observability data collection system using OpenTelemetry, Prometheus, and Loki, eliminating data silos from traditional toolchains and establishing a high-quality multi-source data foundation for smart operations; (2) We proposed a dual-modal workload pattern perception method based on large language models, correctly identifying six workload patterns through text and image fusion analysis, establishing theoretical foundations for predictive resource management; (3) We built a pattern-aware predictive auto-scaling framework that overcomes traditional reactive scaling delays through intelligent model selection and

matching, achieving transformation from passive response to proactive prediction.

Experimental results show our system excels in both workload pattern recognition accuracy and resource prediction precision, providing an effective technical solution for cloud-native system intelligent operations. This research opens new technical pathways for applying large language models in time-series prediction and pattern recognition, possessing significant theoretical and practical value.

## REFERENCES

- [1] Y. N. Zhong, D. Y. Chen, Z. Weng, et al., "Design and implementation of a power cloud platform based on microservice architecture," *Adhesion*, vol. 49, no. 08, pp. 174–176+181, 2022.
- [2] K. Senjab, S. Abbas, N. Ahmed, et al., "A survey of Kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, p. 87, 2023.
- [3] W. Yang, W. Liu, X. Wei, et al., "EdgeKeeper: A trusted edge computing framework for ubiquitous power Internet of Things," *Frontiers of Information Technology & Electronic Engineering*, vol. 22, no. 3, pp. 374–399, 2021.
- [4] Y. B. Liang, B. H. Qian, M. Y. Zhu, et al., "TM-HEDGE: A tracing link reconstruction technology for digital service networks," *Computer Engineering*, vol. 2025, pp. 1–10. [Online]. Available: <https://doi.org/10.19678/j.issn.1000-3428.0070444>.
- [5] V. Raj and S. Ravichandra, "A service graph based extraction of microservices from monolith services of service-oriented architecture," *Software: Practice and Experience*, vol. 52, no. 7, pp. 1661–1678, 2022.
- [6] Y. Yang, Y. Li, and Z. H. Wu, "A survey of distributed tracing technology," *Journal of Software*, vol. 31, no. 07, pp. 2019–2039, 2020. [Online]. Available: <https://doi.org/10.13328/j.cnki.jos.006047>.
- [7] Q. X. Zhang, Y. Wu, Y. Yang, et al., "Survey on service dependency discovery technologies for microservice systems," *Journal of Software*, vol. 35, no. 1, pp. 118–135, 2024.
- [8] V. Lenarduzzi, F. Lomio, N. Saarimäki, et al., "Does migrating a monolithic system to microservices decrease the technical debt?" *Journal of Systems and Software*, vol. 169, p. 110710, 2020.
- [9] B. B. Rad, H. J. Bhatti, and M. Ahmadi, "An introduction to Docker and analysis of its performance," *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 17, no. 3, p. 228, 2017.
- [10] R. Buyya, S. N. Srirama, G. Casale, et al., "A manifesto for future generation cloud computing: Research directions for the next decade," *ACM Computing Surveys (CSUR)*, vol. 51, no. 5, pp. 1–38, 2018.
- [11] A. Botta, "Integration of cloud computing and internet of things: A survey," *Future Generation Computer Systems*, vol. 56, pp. 684–700, 2016.
- [12] B. Burns, B. Grant, D. Oppenheimer, et al., "Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade," *Queue*, vol. 14, no. 1, pp. 70–93, Feb. 2016.
- [13] M. Lukša, *Kubernetes in Action*. New York: Simon and Schuster, 2017.
- [14] A. Vayghan, M. A. Saied, M. Toeroe, et al., "Microservice-based architecture: Towards high-availability for stateful applications with Kubernetes," in *Proc. IEEE 19th Int. Conf. Softw. Quality, Reliability Security (QRS)*, 2019, pp. 176–185.
- [15] X. Z. Tu and H. C. Yang, "A horizontal auto-scaling system based on Kubernetes," *Computer and Modernization*, no. 7, pp. 25–31, 2019.
- [16] S. Shi, Z. H. Wei, X. F. Liu, et al., "Research on an auto-scaling strategy for Kubernetes container cloud based on LSTM," *Manufacturing Automation*, vol. 45, no. 09, pp. 189–196, 2023.
- [17] Chang C C, Yang S R, Yeh E H, et al. A kubernetes-based monitoring platform for dynamic cloud resource provisioning[C]//GLOBECOM 2017-2017 IEEE Global Communications Conference. IEEE, 2017: 1-6.
- [18] Xing Y, Lv Y, Zeng X, et al. SABER: A MAPE-K-based Self-Adaptive Framework for Microservice Bad Smell Refactoring[C]//2025 IEEE International Conference on Web Services (ICWS). IEEE, 2025: 673-684.